

Diberikan skema basis data berikut ini. Atribut dengan garis bawah adalah *primary key*, sedangkan atribut tercetak miring adalah *foreign key* (ke atribut *primary key* dengan nama sama):

SUPPLIER = (SID, SNAME, ADDRESS, SCITY, RATING, BALANCE)

PRODUCT = (PID, PNAME, *CID*, WEIGHT)

PRODUCT_CATEGORY = (CID, CNAME, DESCRIPTION)

SUPPLY = (ORDERDATE, *PID*, *SID*, QUANTITY, PRICE, RECEIVEDATE)

Berdasarkan hasil analisis terhadap *workload* sistem yang telah berjalan dan data yang ada di basis data, diperoleh fakta berikut ini.

- Terdapat 10.000 *record* pada SUPPLIER, 12.000 pada PRODUCT, 1.500.000 pada SUPPLY, dan 100 pada PRODUCT_CATEGORY.
- 90% akses terhadap produk selalu membutuhkan nama kategori dari produk tersebut.
- 80% akses terhadap *supply* hanya mengakses data pesanan dari *supplier* yang belum diterima saja. Setiap saat, rata-rata hanya ada 20.000 hingga 30.000 pesanan dari *supplier* yang belum diterima. Pesanan yang belum diterima ditandai dengan atribut RECEIVEDATE pada suatu baris data pesanan di tabel SUPPLY bernilai NULL.
- Atribut Alamat, Kota, dan Rating dari *supplier* hanya diakses setiap awal bulan, pada saat akan membuat laporan kegiatan pada bulan sebelumnya. Atribut Balance sering sekali berubah, yaitu setiap kali ada pesanan ke *supplier* tersebut.
- Pencarian data *supplier* dan *product* sangat sering dilakukan berdasarkan nama.
- Untuk memenuhi kebutuhan dari pihak eksekutif, *query* untuk mendapatkan total harga terhutang dari seluruh pesanan yang belum diterima, baik per *supplier* maupun per tanggal pesanan, sering dijalankan.

Tugas Anda:

Berikan usulan *schema tuning* untuk meningkatkan kinerja sistem basis data di atas, termasuk kemungkinan untuk memanfaatkan *materialized view* dan kebutuhan akan indeks. Indeks yang tidak perlu dituliskan adalah indeks yang berkaitan dengan *primary key*. Untuk setiap usulan:

- Tuliskan jenis *tuning*-nya dan jelaskan.
- Tuliskan skema baru yang dihasilkan (jika ada).
- Tuliskan kebutuhan proses tambahan di basis data untuk tetap menjaga konsistensi data (jika ada).
- Jika perlu *index*, tuliskan dengan jelas relasi dan atribut indeks dan jenis indeks yang dibutuhkan.
- Untuk setiap *materialized view* yang Anda usulkan, tuliskan perintah *create view* untuk mendapatkannya.

Di akhir jawaban, tuliskan kembali seluruh skema basis data final yang Anda usulkan, termasuk skema *materialized view* (jika ada).

1. Kasus 1: Akses PRODUCT selalu membutuhkan nama kategori

Untuk mengatasi permasalahan “90% akses terhadap produk selalu membutuhkan nama kategori dari produk tersebut”, jenis *tuning* yang digunakan adalah *denormalisasi* (*adding redundant columns*). Diperlukan *denormalisasi* pada tabel PRODUCT dengan menambahkan atribut CNAME dari tabel PRODUCT_CATEGORY.

Skema baru:

PRODUCT = (PID, PNAME, *CID*, CNAME, WEIGHT)

Nilai CNAME pada tabel PRODUCT harus selalu sama dengan nilai CNAME pada PRODUCT_CATEGORY untuk *CID* yang sama. Oleh karena itu, ketika terjadi perubahan CNAME pada PRODUCT_CATEGORY, nilai CNAME pada seluruh PRODUCT dengan *CID* yang sesuai juga harus diperbarui. Proses tersebut dapat dilakukan dengan *trigger* atau mekanisme aplikasi.

2. Kasus 2: Sebagian besar akses SUPPLY hanya membutuhkan pesanan yang belum diterima

Untuk mengatasi permasalahan “80% akses terhadap *supply* hanya mengakses data pesanan dari *supplier* yang belum diterima saja. Setiap saat, rata-rata hanya ada 20.000 hingga 30.000 pesanan dari *supplier* yang belum diterima”, jenis *tuning* yang digunakan adalah *horizontal splitting*.

Diperlukan *horizontal splitting* pada tabel SUPPLY berdasarkan nilai atribut RECEIVEDATE. Tabel SUPPLY_RECEIVED berisi data SUPPLY dengan RECEIVEDATE tidak NULL, sedangkan tabel SUPPLY_NOTRECEIVED berisi data SUPPLY dengan

RECEIVEDATE bernilai NULL.

Skema baru:

SUPPLY_RECEIVED = (ORDERDATE, PID, SID, QUANTITY, PRICE, RECEIVEDATE)

SUPPLY_NOTRECEIVED = (ORDERDATE, PID, SID, QUANTITY, PRICE, RECEIVEDATE)

Data pada SUPPLY_RECEIVED dan SUPPLY_NOTRECEIVED harus disjoint dan jika digabungkan berdasarkan primary key harus menghasilkan seluruh data pada tabel SUPPLY. Selain itu, nilai RECEIVEDATE pada SUPPLY_RECEIVED harus tidak NULL, sedangkan nilai RECEIVEDATE pada SUPPLY_NOTRECEIVED harus NULL.

Lalu, jika pesanan dari supplier yang belum diterima pada tabel SUPPLY_NOTRECEIVED berubah status menjadi diterima, baris data tersebut harus dipindahkan ke SUPPLY_RECEIVED dengan mengisi nilai RECEIVEDATE.

3. **Kasus 3: Atribut SUPPLIER memiliki frekuensi akses dan perubahan yang berbeda**

Untuk mengatasi permasalahan “Atribut Alamat, Kota, dan Rating dari supplier hanya diakses setiap awal bulan, pada saat akan membuat laporan kegiatan pada bulan sebelumnya. Atribut Balance sering sekali berubah, yaitu setiap kali ada pesanan ke supplier tersebut”, jenis tuning yang digunakan adalah vertical splitting. Atribut SADDRESS, SCITY, dan RATING hanya diakses pada waktu tertentu, sedangkan atribut BALANCE sering berubah. Oleh karena itu, tabel SUPPLIER dapat dibagi secara vertikal menjadi tabel utama dan tabel detail.

Skema baru:

SUPPLIER_MAIN = (SID, SNAME, BALANCE)

SUPPLIER_DETAIL = (SID, SADDRESS, SCITY, RATING)

Setiap SID yang terdapat pada SUPPLIER_MAIN harus memiliki data yang sesuai pada SUPPLIER_DETAIL. Ketika supplier baru ditambahkan, data harus ditambahkan ke kedua tabel. Ketika supplier dihapus, data terkait pada kedua tabel juga harus dihapus.

4. **Kasus 4: Pencarian SUPPLIER dan PRODUCT berdasarkan nama**

Untuk mengatasi permasalahan “Pencarian data supplier dan product sangat sering dilakukan berdasarkan nama”, perlu dilakukan tuning dengan jenis index tuning.

a. Supplier

Berikan index pada SUPPLIER_MAIN(SNAME). Jenis index yang digunakan adalah B+-tree index. B+-tree dipilih karena merupakan jenis index yang umum digunakan dan dapat mendukung pencarian berdasarkan nilai atribut secara efisien.

b. Product

Berikan index pada PRODUCT(PNAME). Jenis index yang digunakan adalah

B+-tree index. Index tersebut digunakan karena pencarian data product sangat sering dilakukan berdasarkan nama.

5. Kasus 5: Query total harga pesanan yang belum diterima sering dijalankan

a. Masalah total harga terhutang per supplier

Untuk mengatasi permasalahan “Untuk memenuhi kebutuhan dari pihak eksekutif, query untuk mendapatkan total harga terhutang dari seluruh pesanan yang belum diterima, baik per supplier sering dijalankan”, perlu dibuat materialized view yang menyimpan hasil agregasi total harga per supplier sehingga query yang sama tidak perlu melakukan agregasi ulang terhadap tabel SUPPLY_NOTRECEIVED setiap kali dijalankan.

Skema materialized view:

VHARGAPERSID = (SID, TOTPRICE)

Perintah yang digunakan:

```
CREATE MATERIALIZED VIEW VHARGAPERSID AS
SELECT SID, SUM(QUANTITY * PRICE) AS TOTPRICE
FROM SUPPLY_NOTRECEIVED
GROUP BY SID;
```

b. Masalah total harga terhutang per tanggal pesanan

Untuk mengatasi permasalahan “Untuk memenuhi kebutuhan dari pihak eksekutif, query untuk mendapatkan total harga terhutang dari seluruh pesanan yang belum diterima, baik per tanggal pesanan sering dijalankan”, perlu dibuat materialized view yang menyimpan hasil agregasi total harga berdasarkan tanggal pesanan sehingga query untuk memperoleh total harga terhutang per tanggal dapat dilakukan dengan lebih efisien.

Skema materialized view:

VHARGAPERORDERDATE = (ORDERDATE, TOTPRICE)

Perintah yang digunakan:

```
CREATE MATERIALIZED VIEW VHARGAPERORDERDATE AS
SELECT ORDERDATE, SUM(QUANTITY * PRICE) AS TOTPRICE
FROM SUPPLY_NOTRECEIVED
GROUP BY ORDERDATE;
```

Skema basis data final:

PRODUCT = (PID, PNAME, CID, CNAME, WEIGHT)

PRODUCT_CATEGORY = (CID, CNAME, DESCRIPTION)

SUPPLIER_DETAIL = (SID, SADDRESS, SCITY, RATING)

SUPPLY_RECEIVED = (ORDERDATE, PID, SID, QUANTITY, PRICE, RECEIVEDATE)

SUPPLY_NOTRECEIVED = (ORDERDATE, PID, SID, QUANTITY, PRICE, RECEIVEDATE)

Skema materialized view:

VHARGAPERSID = (SID, TOTPRICE)

VHARGAPERORDERDATE = (ORDERDATE, TOTPRICE)

Index yang digunakan adalah B+ tree index pada:

- SUPPLIER_NAME(SNAME)
- PRODUCT(PNAME)